Recipe recommendation system

ML week

by: CRUX LAB

Overview

- Task Description
- Complexity Levels
- Input Data
- Deliverables
- Results Presentation
- Evaluation Criteria
- Getting Started

Task Description

You're building a specialty coffee recipe app. Users have different equipment, ingredient availability, and taste preferences.

Your goal: Build a recommendation system that predicts which recipes a user will enjoy, using their profile, equipment, and interaction history. Unlike movie recommendations where users rarely re-watch, coffee lovers often want to re-make their favorites. Your system can recommend recipes the user has already tried – the goal is to predict what they'll enjoy now, not just discover new things.

Challenges:

1. **Equipment Constraints**
   Users own different equipment. Can't recommend espresso drinks to someone with only a French press.
2. **Taste Matching**
   Each recipe has a taste profile. Match user preferences for bitterness, sweetness, acidity, and body.
3. **Cold Start**
   New users have no interaction history. How do you recommend without behavioral data?

Complexity Levels

Choose at least one level. Each level is standalone and presentable, but the levels can also be combined in a hybrid approach. Use NDCG@5 metric for assessing recommendation quality. Aim for NDCG@5 > 0.4

1. **Rule-Based Filtering**
   Filter recipes by equipment/products, rank by taste similarity.
   Cosine similarity for taste, Equipment availability check, etc.

2. **Classical ML**
   Collaborative or content-based filtering with hybrid cold-start handling.
   User-based / item-based CF, Matrix factorization (SVD), Feature engineering, etc.

3. **Deep Learning**
   Neural collaborative filtering or embedding-based approaches.
   NCF / Two-tower models, Sequence-aware recs, etc.

Input Data

[Link to Dataset]

- **recipes.csv**
  Recipe catalog with taste profiles, equipment requirements, ingredients
- **users.csv**
  User profiles with owned equipment, taste preferences, dietary restrictions
- **interactions_train.csv**
  Training data: user-recipe interactions with timestamps and ratings
- **interactions_val.csv**
  Validation set for warm users (time-based split) – DO NOT train on this
- **interactions_val_cold.csv**
  Validation set for cold-start users – DO NOT train on this
- **cold_users.json**
  users with ZERO interaction history – for cold-start evaluation

Deliverables

- **Github repository**

- **Presentation**
  Approach, results, demo

- **Demo application**
  Jupyter notebook at minimum, a working web application that can show recommendations for a selected user is the best solution

Rules:

- No external datasets
- No pretrained recommendation models
- No training on test data
- Standard ML libraries OK (sklearn, torch, etc)

Results Presentation

You can choose how to present your results (ordered from minimal to more feature-complete):

1. **Script**
   Create a script that takes user ID and number of recommendations as input and outputs a list of recommended recipes.

2. **Jupyter Notebook**
   Create an interactive notebook that allows you to specify user ID and number of recommendations and get a list of recommendations. Add explanations and documentation as you see fit.

3. **API**
   Create a server with an endpoint that receives user ID and number of recommendations and responds with a list of recommendations. Think about data storage.

4. **API + Web app**
   In addition to an API endpoint, create a web page where, after selecting a user and entering the number of recommendations, you are presented with this user's past interactions and a list of recommended recipes

Evaluation Criteria

1. **Methodology and Complexity – 25%**
   Sound approach, justified decisions
2. **Completeness – 20%**
   All chosen levels fully implemented
3. **Presentation – 20%**
   Clear explanation, demo quality
4. **Code Quality – 20%**
   Clean code, documentation
5. **Creativity – 15%**
   Novel ideas, edge case handling

Bonus points (+5% each):

1. Cold-start handling (tested separately)
2. Explainability ("why this recipe?")
3. Inference speed optimization (aim for < 50ms per user)
4. Working demo application

CRUX LAB

Getting Started

- Google's Recommendation Systems Course https://developers.google.com/machine-learning/recommendation
- Cosine Similarity for Recommendation Systems https://www.learndatasci.com/glossary/cosine-similarity/
- SVD for Recommendations https://surprise.readthedocs.io/en/stable/matrix_factorization.html
- PyTorch Recommendation Tutorial https://docs.pytorch.org/tutorials/intermediate/torchrec_intro_tutorial.html
- LightGBM for LTR Recommendations https://lightgbm.readthedocs.io/en/latest/index.html
- Demystifying NDCG https://towardsdatascience.com/demystifying-ndcg-bee3be58cfe0/

Useful Libraries for Python
- Essentials: pandas, numpy, scikit-learn
- Recommendations-specific: surprise, implicit, lightgbm, torch

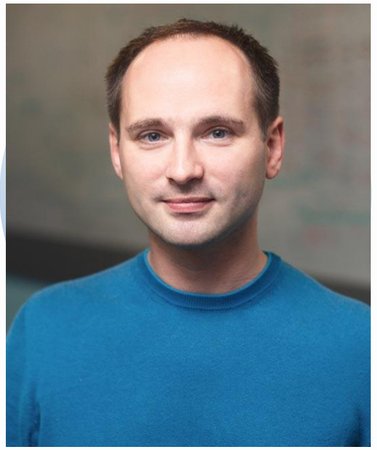
Tips:
- Filter for equipment: users can only make recipes they have equipment for
- Handle cold-start explicitly
- Combine approaches: hybrid models often work best

THANK YOU!

Igor Ishchenko
CTO & Co-founder

Cruxlab, Inc

https://cruxlab.com



Oleg Konovalov
Tech Lead

Cruxlab, Inc

https://cruxlab.com